

TUGAS PRAKTIKUM 6: DEADLOCK



Dosen Pengampu:

Triawan Adi Cahyanto M.Kom

Disusun Oleh:

Nabila Senja Ramadhani (2410651043)

PROGRAM STUDI TEKNIK INFORMATIKA

FAKULTAS TEKNIK

UNIVERSITAS MUHAMMADIYAH JEMBER

2025

Tugas 1: Analisis Deadlock

Jalankan program deadlock_simple.c dan jelaskan:

```
senjaramadhani@LAPTOP-823QCJCF:~$ nano deadlock_simple.c
senjaramadhani@LAPTOP-823QCJCF:~$ gcc deadlock_simple.c -o deadlock_simple -lpthread
senjaramadhani@LAPTOP-823QCJCF:~$ ./deadlock_simple
Thread 1: Mencoba mengambil Lock 1...
Thread 1: Berhasil dapat Lock 1!
Thread 2: Mencoba mengambil Lock 2...
Thread 2: Berhasil dapat Lock 2!
Thread 2: Mencoba mengambil Lock 1...
Thread 1: Mencoba mengambil Lock 2...
^C
senjaramadhani@LAPTOP-823QCJCF:~$ |
```

1. Mengapa program mengalami deadlock?

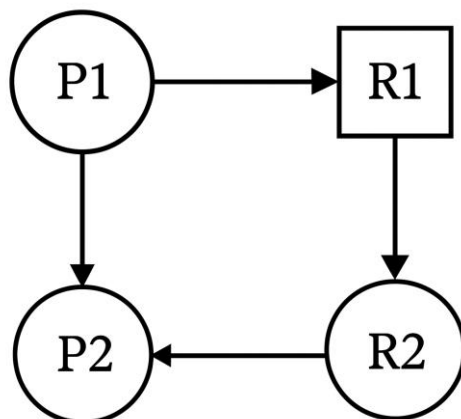
Deadlock terjadi karena Thread 1 memegang Lock A kemudian menunggu Lock B, sementara Thread 2 memegang Lock B dan menunggu Lock A. Keduanya tidak pernah melepaskan lock sehingga proses berhenti sepenuhnya.

2. Kondisi deadlock apa saja yang terpenuhi?

Empat kondisi deadlock (Coffman Conditions) yang terjadi:

- Mutual Exclusion – Lock hanya dapat dipegang oleh satu thread pada satu waktu.
- Hold and Wait – Thread 1 memegang Lock A sambil menunggu Lock B; Thread 2 memegang Lock B sambil menunggu Lock A.
- No Preemption – Lock tidak dapat dipaksa dilepas; hanya pemegang lock yang bisa melepas.
- Circular Wait – $T1 \rightarrow A \rightarrow B \rightarrow T2 \rightarrow A$ membentuk lingkaran.

3. Gambar diagram resource allocation graph-nya



Tugas 2: Implementasi Pencegahan

Modifikasi program deadlock_simple.c menggunakan teknik:

1. Lock ordering

Deadlock dapat dicegah dengan memastikan bahwa seluruh thread mengambil lock dalam urutan yang sama.

```
senjaramadhani@LAPTOP-823QCJCF:~$ nano prevention_ordering.c
senjaramadhani@LAPTOP-823QCJCF:~$ gcc prevention_ordering.c -o prevention_ordering -lpthread
senjaramadhani@LAPTOP-823QCJCF:~$ ./prevention_ordering
Thread mulai kerja..
Mengambil Lock 1
Mengambil Lock 2
Thread memproses data...
Thread mulai kerja..
Thread selesai.
Mengambil Lock 1
Mengambil Lock 2
Thread memproses data...
Thread selesai.
senjaramadhani@LAPTOP-823QCJCF:~$ |
```

Tidak terjadi deadlock karena kedua thread mengikuti urutan lock yang sama. Thread akan saling menunggu (blocking), tetapi sistem tetap berjalan

Source code:

```
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>
```

```
pthread_mutex_t lock1 = PTHREAD_MUTEX_INITIALIZER;
pthread_mutex_t lock2 = PTHREAD_MUTEX_INITIALIZER;
```

```
void acquire_in_order() {
    pthread_mutex_lock(&lock1);
    printf("Mengambil Lock 1\n");

    pthread_mutex_lock(&lock2);
    printf("Mengambil Lock 2\n");
}
```

```
void release_in_order() {
    pthread_mutex_unlock(&lock2);
    pthread_mutex_unlock(&lock1);
}
```

```
void* thread_func(void* arg) {
    printf("Thread mulai kerja..\n");

    acquire_in_order();
    printf("Thread memproses data...\n");
    sleep(1);
    release_in_order();

    printf("Thread selesai.\n");
    return NULL;
}
```

```
int main() {
```

```

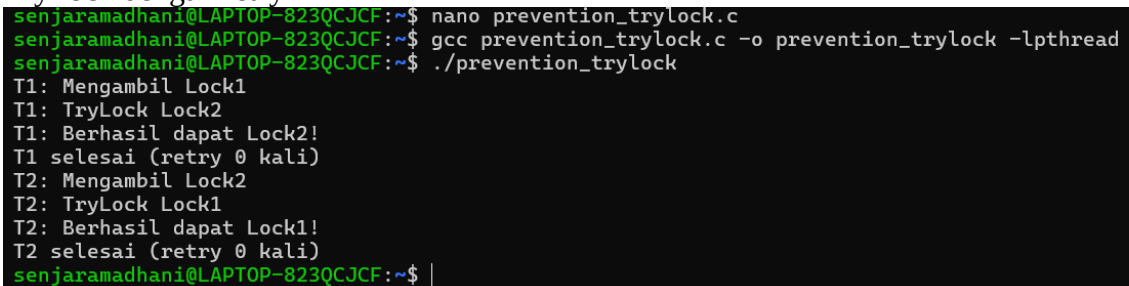
pthread_t t1, t2;

pthread_create(&t1, NULL, thread_func, NULL);
pthread_create(&t2, NULL, thread_func, NULL);

pthread_join(t1, NULL);
pthread_join(t2, NULL);
return 0;
}

```

2. Try-lock dengan retry



```

senjaramadhani@LAPTOP-823QCJCF:~$ nano prevention_trylock.c
senjaramadhani@LAPTOP-823QCJCF:~$ gcc prevention_trylock.c -o prevention_trylock -lpthread
senjaramadhani@LAPTOP-823QCJCF:~$ ./prevention_trylock
T1: Mengambil Lock1
T1: TryLock Lock2
T1: Berhasil dapat Lock2!
T1 selesai (retry 0 kali)
T2: Mengambil Lock2
T2: TryLock Lock1
T2: Berhasil dapat Lock1!
T2 selesai (retry 0 kali)
senjaramadhani@LAPTOP-823QCJCF:~$ |

```

Tidak terjadi deadlock. Namun proses dapat membutuhkan retry berkali-kali dan dapat menambah waktu eksekusi.

Source code:

```

#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

```

```

pthread_mutex_t lock1 = PTHREAD_MUTEX_INITIALIZER;
pthread_mutex_t lock2 = PTHREAD_MUTEX_INITIALIZER;

```

```

void* thread1_func(void* arg) {
    int retry = 0;

    while (1) {
        pthread_mutex_lock(&lock1);
        printf("T1: Mengambil Lock1\n");

        printf("T1: TryLock Lock2\n");
        if (pthread_mutex_trylock(&lock2) == 0) {
            printf("T1: Berhasil dapat Lock2!\n");
            sleep(1);
            pthread_mutex_unlock(&lock2);
            pthread_mutex_unlock(&lock1);
            break;
        } else {
            printf("T1: Gagal, melepas Lock1 dan retry...\n");
            pthread_mutex_unlock(&lock1);
            retry++;
            usleep(100000);
        }
    }
}

```

```

    printf("T1 selesai (retry %d kali)\n", retry);
    return NULL;
}

void* thread2_func(void* arg) {
    int retry = 0;

    while (1) {
        pthread_mutex_lock(&lock2);
        printf("T2: Mengambil Lock2\n");

        printf("T2: TryLock Lock1\n");
        if (pthread_mutex_trylock(&lock1) == 0) {
            printf("T2: Berhasil dapat Lock1!\n");
            sleep(1);
            pthread_mutex_unlock(&lock1);
            pthread_mutex_unlock(&lock2);
            break;
        } else {
            printf("T2: Gagal, melepas Lock2 dan retry...\n");
            pthread_mutex_unlock(&lock2);
            retry++;
            usleep(150000);
        }
    }

    printf("T2 selesai (retry %d kali)\n", retry);
    return NULL;
}

int main() {
    pthread_t t1, t2;
    pthread_create(&t1, NULL, thread1_func, NULL);
    pthread_create(&t2, NULL, thread2_func, NULL);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);
    return 0;
}

```

3. Bandingkan kedua metode tersebut

Aspek	Lock Ordering	Try-Lock & Retry
Deadlock	100% mencegah	100% mencegah
Blocking	Bisa terjadi	Minim blocking
Retry	Tidak ada	Ada
Kestabilan	Sangat stabil	Kurang stabil jika banyak thread
Kasus penggunaan	Sistem besar, terstruktur	Sistem dengan konflik lock tinggi

Tugas 3: Banker's Algorithm

Buat program baru dengan data:

- 4 proses
- 3 jenis resource
- Available: [5, 4, 3]
- Implementasikan request resource dan cek keamanan sistem

```
senjaramadhani@LAPTOP-823QCJCF:~$ nano bankers_algorithm.c
senjaramadhani@LAPTOP-823QCJCF:~$ gcc bankers_algorithm.c -o bankers_algorithm
senjaramadhani@LAPTOP-823QCJCF:~$ ./bankers_algorithm
=== BANKER'S ALGORITHM ===
Available: 5 4 3

=== REQUEST PROSES P0 ===
Request: 1 0 1

[CHECK] Mengecek apakah sistem dalam keadaan aman...
[HASIL] Sistem AMAN. Safe Sequence: P0 P1 P2 P3
[DITERIMA] Request disetujui!

=== REQUEST PROSES P3 ===
Request: 2 2 1

[CHECK] Mengecek apakah sistem dalam keadaan aman...
[HASIL] Sistem TIDAK AMAN (unsafe)
[DIBATALKAN] Request membuat sistem tidak aman. Rollback.
senjaramadhani@LAPTOP-823QCJCF:~$ |
```

Source code:

```
#include <stdio.h>
```

```
#include <stdbool.h>
```

```
#define P 4 // Jumlah proses
```

```
#define R 3 // Jumlah resource
```

```
// Available resources
```

```
int available[R] = {5, 4, 3};
```

```
// Maximum need tiap proses
```

```
int maximum[P][R] = {
```

```
    {4, 3, 2},
```

```
    {2, 2, 2},
```

```
    {3, 3, 3},
```

```
    {5, 4, 3}
```

```
};
```

```
// Resource yang sudah dialokasikan
```

```
int allocation[P][R] = {
```

```
    {1, 0, 1},
```

```
    {1, 1, 0},
```

```
    {1, 2, 1},
```

```
    {0, 1, 1}
```

```
};
```

```
// Need = Maximum - Allocation
```

```

int need[P][R];

void calculate_need() {
    for (int i = 0; i < P; i++) {
        for (int j = 0; j < R; j++) {
            need[i][j] = maximum[i][j] - allocation[i][j];
        }
    }
}

bool is_safe() {
    int work[R];
    bool finish[P] = {false};
    int safeSeq[P];
    int count = 0;

    for (int i = 0; i < R; i++)
        work[i] = available[i];

    printf("\n[CHECK] Mengecek apakah sistem dalam keadaan aman...\n");

    while (count < P) {
        bool found = false;

        for (int i = 0; i < P; i++) {
            if (!finish[i]) {
                bool canRun = true;

                for (int j = 0; j < R; j++) {
                    if (need[i][j] > work[j]) {
                        canRun = false;
                        break;
                    }
                }

                if (canRun) {
                    for (int j = 0; j < R; j++)
                        work[j] += allocation[i][j];

                    safeSeq[count++] = i;
                    finish[i] = true;
                    found = true;
                }
            }
        }

        if (!found) {
            printf("[HASIL] Sistem TIDAK AMAN (unsafe)\n");
            return false;
        }
    }
}

```

```

    }

    printf("[HASIL] Sistem AMAN. Safe Sequence: ");
    for (int i = 0; i < P; i++)
        printf("P%d ", safeSeq[i]);
    printf("\n");

    return true;
}

bool request_resource(int process, int request[]) {
    printf("\n=== REQUEST PROSES P%d ===\n", process);
    printf("Request: ");
    for (int i = 0; i < R; i++)
        printf("%d ", request[i]);
    printf("\n");

    // 1. Cek request ≤ need
    for (int i = 0; i < R; i++) {
        if (request[i] > need[process][i]) {
            printf("[DITOLAK] Request melebihi need!\n");
            return false;
        }
    }

    // 2. Cek request ≤ available
    for (int i = 0; i < R; i++) {
        if (request[i] > available[i]) {
            printf("[DITOLAK] Resource tidak cukup tersedia!\n");
            return false;
        }
    }

    // 3. Lakukan simulasi alokasi
    for (int i = 0; i < R; i++) {
        available[i] -= request[i];
        allocation[process][i] += request[i];
        need[process][i] -= request[i];
    }

    // 4. Cek apakah sistem masih safe
    if (is_safe()) {
        printf("[DITERIMA] Request disetujui!\n");
        return true;
    }

    // Jika tidak aman → rollback
    printf("[DIBATALKAN] Request membuat sistem tidak aman. Rollback.\n");
    for (int i = 0; i < R; i++) {
        available[i] += request[i];
    }
}

```

```

        allocation[process][i] -= request[i];
        need[process][i] += request[i];
    }

    return false;
}

int main() {
    calculate_need();

    printf("=== BANKER'S ALGORITHM ===\n");
    printf("Available: %d %d %d\n\n", available[0], available[1], available[2]);

    int req1[R] = {1, 0, 1};
    request_resource(0, req1);

    int req2[R] = {2, 2, 1};
    request_resource(3, req2);

    return 0;
}

```

Tugas 4: Kasus Nyata (Real Case)

Buat simulasi deadlock untuk kasus: "Dua orang ingin transfer uang antar rekening secara bersamaan"

- Orang A: transfer dari Rekening 1 ke Rekening 2
- Orang B: transfer dari Rekening 2 ke Rekening 1
- Implementasikan deadlock prevention

```

senjaramadhani@LAPTOP-823QCJCF:~$ nano lock_ordering.c
senjaramadhani@LAPTOP-823QCJCF:~$ gcc lock_ordering.c -o lock_ordering -lpthread
senjaramadhani@LAPTOP-823QCJCF:~$ ./lock_ordering
A: Lock r1 → r2
A: Transfer 1 → 2 selesai
B: Lock r1 → r2
B: Transfer 2 → 1 selesai
senjaramadhani@LAPTOP-823QCJCF:~$ nano trylock_retry.c
senjaramadhani@LAPTOP-823QCJCF:~$ gcc trylock_retry.c -o trylock_retry -lpthread
senjaramadhani@LAPTOP-823QCJCF:~$ ./trylock_retry
A: Lock Rekening 1
A: Try-lock Rekening 2
A: Transfer 1 → 2 selesai
B: Lock Rekening 2
B: Try-lock Rekening 1
B: Transfer 2 → 1 selesai
senjaramadhani@LAPTOP-823QCJCF:~$ nano deadlock_transfer.c
senjaramadhani@LAPTOP-823QCJCF:~$ gcc deadlock_transfer.c -o deadlock_transfer -lpthread
senjaramadhani@LAPTOP-823QCJCF:~$ ./deadlock_transfer
A: Mengunci Rekening 1
B: Mengunci Rekening 2
A: Mencoba mengunci Rekening 2
B: Mencoba mengunci Rekening 1
^C
senjaramadhani@LAPTOP-823QCJCF:~$ |

```

Lock Ordering source code:

```
#include <stdio.h>
```

```
#include <pthread.h>
```

```
pthread_mutex_t r1 = PTHREAD_MUTEX_INITIALIZER;
```

```
pthread_mutex_t r2 = PTHREAD_MUTEX_INITIALIZER;
```

```

void lock_ordered(pthread_mutex_t* first, pthread_mutex_t* second) {
    pthread_mutex_lock(first);
    pthread_mutex_lock(second);
}

void unlock_ordered(pthread_mutex_t* first, pthread_mutex_t* second) {
    pthread_mutex_unlock(second);
    pthread_mutex_unlock(first);
}

void* transfer_A(void* arg) {
    printf("A: Lock r1 → r2\n");
    lock_ordered(&r1, &r2);

    printf("A: Transfer 1 → 2 selesai\n");
    unlock_ordered(&r1, &r2);
    return NULL;
}

void* transfer_B(void* arg) {
    printf("B: Lock r1 → r2\n");
    lock_ordered(&r1, &r2);

    printf("B: Transfer 2 → 1 selesai\n");
    unlock_ordered(&r1, &r2);
    return NULL;
}

int main() {
    pthread_t t1, t2;

    pthread_create(&t1, NULL, transfer_A, NULL);
    pthread_create(&t2, NULL, transfer_B, NULL);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);
}

```

Try-Lock + Retry source code:

```

#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

pthread_mutex_t r1 = PTHREAD_MUTEX_INITIALIZER;
pthread_mutex_t r2 = PTHREAD_MUTEX_INITIALIZER;

void* transfer_A(void* arg) {
    while (1) {
        printf("A: Lock Rekening 1\n");
        pthread_mutex_lock(&r1);
    }
}

```

```

    printf("A: Try-lock Rekening 2\n");
    if (pthread_mutex_trylock(&r2) == 0) {
        printf("A: Transfer 1 → 2 selesai\n");
        pthread_mutex_unlock(&r2);
        pthread_mutex_unlock(&r1);
        break;
    }

    printf("A: Gagal → retry\n");
    pthread_mutex_unlock(&r1);
    usleep(100000);
}
return NULL;
}

void* transfer_B(void* arg) {
    while (1) {
        printf("B: Lock Rekening 2\n");
        pthread_mutex_lock(&r2);

        printf("B: Try-lock Rekening 1\n");
        if (pthread_mutex_trylock(&r1) == 0) {
            printf("B: Transfer 2 → 1 selesai\n");
            pthread_mutex_unlock(&r1);
            pthread_mutex_unlock(&r2);
            break;
        }

        printf("B: Gagal → retry\n");
        pthread_mutex_unlock(&r2);
        usleep(120000);
    }
    return NULL;
}

int main() {
    pthread_t t1, t2;

    pthread_create(&t1, NULL, transfer_A, NULL);
    pthread_create(&t2, NULL, transfer_B, NULL);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);
}

```